Experiment No. 4

Title: To Develop Distributed Applications Using Client-Server Communication through Java Sockets and RMI Techniques

1. Aim

To develop distributed applications using client-server communication by implementing communication programs based on Java Socket Programming and Remote Method Invocation (RMI) techniques.

2. Objectives

After completing this experiment, students will be able to:

1. Understand the concept of distributed applications.
2. Understand the client-server communication model.
3. Implement communication using TCP sockets in Java.
4. Develop a Java Socket Server and Socket Client.
5. Understand the architecture and working of Java RMI.
6. Develop and invoke remote methods using Java RMI.
7. Compare Socket-based communication with RMI-based communication.
8. Apply distributed communication concepts to real-world applications.

3. Prerequisite Knowledge

Students should have knowledge of:

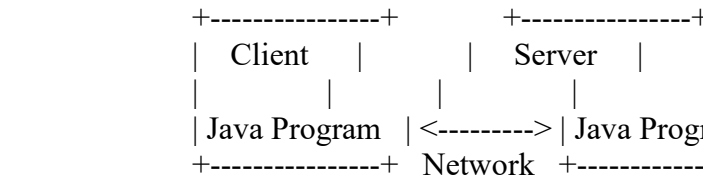
- Java programming
- Classes and objects
- Inheritance and interfaces
- Exception handling
- Input/Output streams
- Networking fundamentals
- IP address and port number
- Client-server architecture
- Multithreading basics

4. Theory

4.1 Distributed Application

A distributed application is an application whose components execute on multiple computers or processes and communicate with each other over a network.

For example:



The client sends a request to the server, and the server processes the request and sends a response.

---

5. Client-Server Model

The client-server model consists of two major components:

Client

The client:

- Initiates communication.

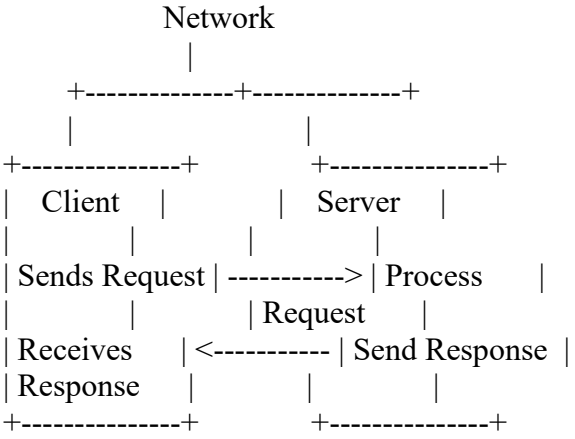
- Sends requests.
- Receives responses.

Server

The server:

- Waits for client requests.
- Accepts client connections.
- Processes requests.
- Sends responses.

Architecture



Part A – Java Socket Programming

6. Java Socket Programming

A socket is an endpoint for communication between two machines over a network.

Java provides the following important classes:

Class	Purpose
Socket	Establishes client-server connection
ServerSocket	Creates a server and listens for connections
InputStream	Reads data
OutputStream	Sends data
BufferedReader	Reads text efficiently
PrintWriter	Sends formatted text

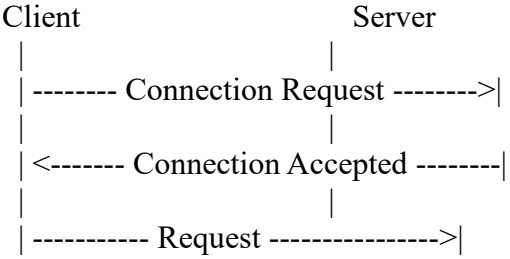
7. TCP Communication

For this experiment, we use TCP (Transmission Control Protocol).

TCP provides:

- Reliable communication
- Connection-oriented communication
- Ordered data delivery
- Error detection
- Retransmission of lost data

Basic TCP communication:



```
|                              |
|<---------- Response ----------|
|                              |
```

8. Problem Statement – Socket
Develop a Java client-server application using TCP Socket Programming where:

1. The server listens on a specified port.
2. The client connects to the server.
3. The client sends a message.
4. The server receives the message.
5. The server processes the message.
6. The server sends a response.
7. The client displays the response.

9. Algorithm – Socket Server
Step 1 Start the server.
Step 2 Create a ServerSocket using a port number.
Step 3 Wait for a client connection using accept().
Step 4 Create input and output streams.
Step 5 Receive the client's message.
Step 6 Process the message.
Step 7 Send the response to the client.
Step 8 Close the connection.
Step 9 Stop the server.

10. Algorithm – Socket Client
Step 1 Start the client.
Step 2 Create a Socket using the server IP address and port number.
Step 3 Create input and output streams.
Step 4 Send a message to the server.
Step 5 Receive the server response.
Step 6 Display the response.
Step 7 Close the socket.

11. Java Socket Server Program
Save as:
SocketServer.java

```
import java.io.*;
import java.net.*;

public class SocketServer {

    public static void main(String[] args) {

        int port = 5000;

        try {

            // Create server socket
            ServerSocket serverSocket =
```

```
            new ServerSocket(port);

        System.out.println(
            "Server started...");

        System.out.println(
            "Waiting for client connection...");

        // Accept client connection
        Socket socket =
            serverSocket.accept();

        System.out.println(
            "Client connected.");

        // Create input stream
        BufferedReader input =
            new BufferedReader(
                new InputStreamReader(
                    socket.getInputStream()));

        // Create output stream
        PrintWriter output =
            new PrintWriter(
                socket.getOutputStream(),
                true);

        // Receive message
        String message =
            input.readLine();

        System.out.println(
            "Client says: " + message);

        // Send response
        output.println(
            "Message received successfully by Server.");

        // Close resources
        input.close();
        output.close();
        socket.close();
        serverSocket.close();

    } catch (IOException e) {

        System.out.println(
            "Server Error: " + e);
    }
}
```

```
}
```

12. Java Socket Client Program
Save as:
SocketClient.java

```
import java.io.*;
import java.net.*;

public class SocketClient {

    public static void main(String[] args) {

        String serverAddress = "localhost";
        int port = 5000;

        try {

            // Connect to server
            Socket socket =
                new Socket(serverAddress, port);

            System.out.println(
                "Connected to server.");

            // Create output stream
            PrintWriter output =
                new PrintWriter(
                    socket.getOutputStream(),
                    true);

            // Create input stream
            BufferedReader input =
                new BufferedReader(
                    new InputStreamReader(
                        socket.getInputStream()));

            // Send message
            output.println(
                "Hello Server, this is the Client.");

            // Receive response
            String response =
                input.readLine();

            System.out.println(
                "Server Response: " + response);

            // Close resources
            output.close();
            input.close();
```

```
            socket.close();

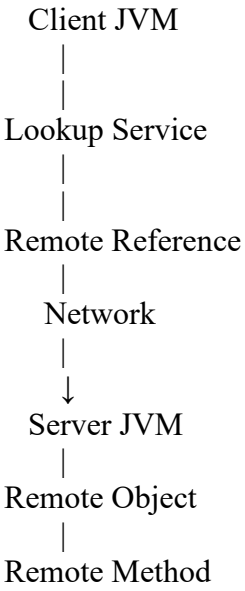
        } catch (IOException e) {

            System.out.println(
                  "Client Error: " + e);
        }
    }
}
```

13. Socket Execution
Terminal 1 – Server
Compile:
javac SocketServer.java
Run:
java SocketServer
Output:
Server started...
Waiting for client connection...
Terminal 2 – Client
Compile:
javac SocketClient.java
Run:
java SocketClient
Client output:
Connected to server.
Server Response: Message received successfully by Server.
Server output:
Server started...
Waiting for client connection...
Client connected.
Client says: Hello Server, this is the Client.

Part B – Java RMI
14. Java RMI
RMI stands for Remote Method Invocation.
Java RMI allows an object running in one JVM to invoke a method on an object running in another JVM.
In simple words:
RMI allows a Java program to call a method of an object located on another machine as if it were a local method.

15. RMI Architecture



16. Components of RMI

The major components are:

1. Remote Interface
2. Remote Object Implementation
3. RMI Registry
4. RMI Server
5. RMI Client

17. Problem Statement – RMI

Develop a Java RMI application where:

- The server provides a remote method.
- The client looks up the remote object.
- The client invokes the remote method.
- The server executes the method.
- The result is returned to the client.

We will implement a simple Remote String Analyzer.

The client sends a string to the server, and the server returns the number of characters in the string.

18. Step 1 – Remote Interface

Save as:

StringAnalyzer.java

```
import java.rmi.*;

public interface StringAnalyzer extends Remote {

   int countCharacters(String str)
         throws RemoteException;
}
```

Explanation

extends Remote

indicates that this interface contains methods that can be invoked remotely.

int countCharacters(String str)
This is the remote method.

throws RemoteException
Remote communication can fail because of network or server problems, so RMI methods must handle RemoteException.

19. Step 2 – Remote Object Implementation
Save as:
StringAnalyzerImpl.java

```
import java.rmi.*;
import java.rmi.server.*;

public class StringAnalyzerImpl
      extends UnicastRemoteObject
      implements StringAnalyzer {

   public StringAnalyzerImpl()
         throws RemoteException {
      super();
   }

   @Override
   public int countCharacters(String str)
         throws RemoteException {

      return str.length();
   }
}
```

20. Explanation
UnicastRemoteObject
extends UnicastRemoteObject
makes the object available for remote invocation.

Implement Interface
implements StringAnalyzer
means that the class provides the implementation of the remote method.

Remote Method
public int countCharacters(String str)
receives a string and returns its length.
For example:
Input: Distributed
Output: 11

23. Step 3 – RMI Server
Save as:
RMIServer.java

```
import java.rmi.registry.*;

public class RMIServer {

   public static void main(String[] args) {

      try {

         // Create remote object
         StringAnalyzerImpl obj =
               new StringAnalyzerImpl();

         // Create RMI Registry
         Registry registry =
               LocateRegistry.createRegistry(1099);

         // Bind remote object
         registry.rebind(
               "StringAnalyzer",
               obj);

         System.out.println(
               "RMI Server is ready...");

      } catch (Exception e) {

         System.out.println(
               "Server Error: " + e);
      }
   }
}
```

21. Important RMI Server Concepts

Create Remote Object

```
StringAnalyzerImpl obj =
      new StringAnalyzerImpl();
```

Creates the remote object.

Create Registry

```
LocateRegistry.createRegistry(1099);
```

Creates the RMI registry on port 1099.
The registry stores references to remote objects.

Bind Object

```
registry.rebind(
      "StringAnalyzer",
      obj);
```

Registers the remote object with the name:
StringAnalyzer
The client uses this name to locate the remote object.

22. Step 4 – RMI Client
Save as:
RMIClient.java

```
import java.rmi.registry.*;

public class RMIClient {

   public static void main(String[] args) {

      try {

         // Connect to RMI Registry
         Registry registry =
               LocateRegistry.getRegistry(
                     "localhost",
                     1099);

         // Lookup remote object
         StringAnalyzer analyzer =
               (StringAnalyzer)
               registry.lookup(
                     "StringAnalyzer");

         // Invoke remote method
         String text =
               "Parallel Computing";

         int result =
               analyzer.countCharacters(text);

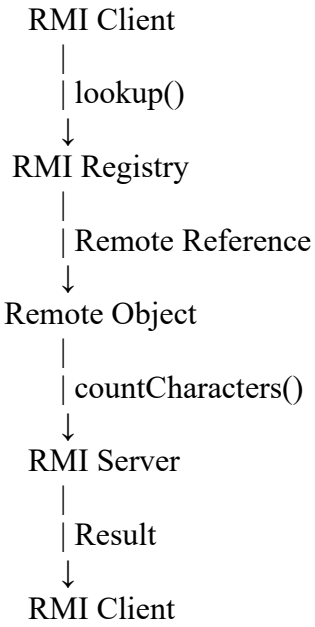
         System.out.println(
               "String: " + text);

         System.out.println(
               "Number of Characters: "
               + result);

      } catch (Exception e) {

         System.out.println(
               "Client Error: " + e);
      }
   }
}
```

23. RMI Execution Flow



24. Compilation and Execution
Compile all files:
javac *.java
Start the RMI server:
java RMIServer
Expected output:
RMI Server is ready...
Open another terminal and run:
java RMIClient
Expected output:
String: Parallel Computing
Number of Characters: 19
The exact count includes the space between the words.

25. Comparison: Socket vs RMI

Feature	Java Socket	Java RMI
Communication	Network-based	Network-based
Programming level	Low-level	Higher-level
Communication	Stream/message based	Method invocation
Main classes	Socket, ServerSocket	Remote, UnicastRemoteObject
Data handling	Programmer manages streams	RMI handles remote invocation
Object-oriented	Less abstract	Highly object-oriented
Registry	Not required	RMI Registry required
Ease of implementation	Moderate	Easier for Java-to-Java applications
Protocol details	Developer handles more details	RMI abstracts many details
Best suited for	General network communication	Distributed Java applications

26. Socket vs RMI Architecture

Socket

Client
 |
 | Socket
 ↓
Server
 |
 | Process data
 ↓
Response

RMI

Client
 |
 | Remote Method Call
 ↓
RMI Registry
 |
 ↓
Remote Object
 |
 | Execute Method
 ↓
Return Result

27. Applications

Socket Programming

- Chat applications
- File transfer
- Multiplayer games
- Client-server applications
- Network monitoring
- Messaging applications

RMI

- Distributed Java applications
- Remote object management
- Distributed computing
- Enterprise applications
- Remote service invocation

Conclusion

Thus, client-server communication programs were successfully developed using Java Socket Programming and Java RMI, demonstrating network communication and remote method invocation in a distributed computing environment.